

LEAF CONDITION ANALYZER

A Bring Your Own Project (BYOP) Report for Computer Vision

Submitted by	Shivam Saini
Registration number	24BAI10026
Course	Computer Vision
Institution	VIT Bhopal University
Project type	Command line computer vision application

An explainable image analysis system that screens a leaf photograph for visible green, yellow, and brown colour patterns.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives and Scope
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Results and Screenshots
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

1. Introduction

Plant leaves can show visible yellowing and browning when affected by stress, pests, nutrient deficiencies, or disease. This project applies basic computer vision techniques to make visual inspection more consistent. Given an image, it estimates green, yellow, and brown portions of a leaf, assigns a transparent condition category, and produces an annotated output image.

2. Problem Statement

Manual leaf inspection is subjective and difficult to scale. A lightweight image processing tool is needed to quantify visible colour patterns and provide an immediate, explainable screening result. The project does not diagnose named plant diseases; it provides a first level visual classification only.

3. Objectives and Scope

The objective is to build a modular, terminal executable application that accepts a leaf image, segments plant like colours in HSV space, measures colour area percentages, classifies visible condition, and saves an annotated result. Included outcomes are mostly healthy looking, possible stress or early disease, likely diseased or severely stressed, and no leaf region detected. Species recognition and named disease diagnosis are outside the scope.

4. Functional Requirements

ID	Requirement	Module
FR-01	Accept a JPG, JPEG, or PNG path through a command line argument.	Input and validation
FR-02	Generate healthy, stressed, and diseased style sample images.	Sample generator
FR-03	Convert BGR to HSV and create green, yellow, and brown masks.	Segmentation
FR-04	Calculate colour percentages and assign a condition category.	Classification
FR-05	Save an annotated output image and print readable or JSON results.	Reporting
FR-06	Reject unreadable, empty, or no leaf inputs with clear errors.	Validation

5. Non functional Requirements

Category	Requirement and strategy
Usability	One command runs the analyser; command line help describes required arguments.
Reliability	Input validation handles missing, unreadable, and no leaf images without a crash.
Maintainability	Analysis, command line handling, sample generation, and tests are separated into focused files.
Performance	Uses array operations and HSV masks; no model download or GPU is required.
Resource efficiency	Processes one image in memory and writes only the requested annotated output.
Testability	Automated tests cover healthy, brown, and blank image behaviours.

6. System Architecture

The design separates user interaction from image analysis logic. The CLI validates arguments and delegates image processing to the analyser. The analyser returns structured metrics and an annotated image; the CLI writes the image and displays the result.

User / Terminal
src/main.py Argument parsing, validation, console or JSON output
src/analyzer.py HSV masks, morphology, percentages, rules, annotation
Input image (.jpg/.png) -> classification + annotated PNG
tests/test_analyzer.py and src/generate_demo_images.py Verification and reproducible demo assets

7. Design Diagrams

Use Case Diagram (text form)

Actor	Use cases
User	Generate sample images; provide an image path; request JSON output; inspect annotated output; run tests.
System	Validate input; segment HSV colours; calculate percentages; classify; save result; report errors.

Workflow Diagram

Start -> provide image path -> validate image -> convert BGR to HSV -> create colour masks -> clean leaf mask -> calculate percentages -> apply rules -> print and save output -> End

Component Diagram

main.py -> analyze_path(path) -> analyze_image(image) -> AnalysisResult
main.py -> cv2.imwrite(output, annotated image)
generate_demo_images.py -> samples/*
test_analyzer.py -> analyze_image(image) -> assertions

Sequence Diagram

Step	User	CLI	Analyzer	File system
1	Runs command	Reads --image and --output		
2		Calls analyze_path	Loads image and validates it	Reads input image
3			Converts HSV; masks and measures	
4		Receives result and annotation		Writes annotated PNG
5	Reads classification	Prints values		

Storage Design

No database is required. The program reads an image from a supplied path and writes an annotated PNG to the requested results path. An ER diagram is therefore not applicable.

8. Design Decisions and Rationale

Decision	Rationale
HSV instead of BGR	Hue identifies colour independently from much of the brightness information, making thresholds more interpretable.
Rule based classification	Thresholds let an evaluator trace each category back to measured colour percentages.
Morphological opening	Small isolated pixels are removed before measurement, reducing mask noise.
Command line interface	Meets terminal execution requirements without a GUI dependency.
Synthetic sample generator	Makes the workflow reproducible without an external dataset.

9. Implementation Details

The implementation uses `src/analyzer.py` for HSV masking, morphology, measurements, decision rules, and annotation; `src/main.py` for the command line interface; `src/generate_demo_images.py` for reproducible inputs; and `tests/test_analyzer.py` for validations. `requirements.txt` declares dependencies.

Execution commands

```
python -m pip install -r requirements.txt
python -m src.generate_demo_images
python -m src.main --image samples/healthy_leaf.png --output results/healthy_annotated.png
```

Classification logic

Visible condition	Decision rule
Likely diseased / severely stressed	brown $\geq$ 20% OR (yellow + brown) $\geq$ 45%
Possible stress / early disease	(yellow + brown) $\geq$ 15%
Mostly healthy looking	Otherwise, if a leaf region is detected
No leaf region detected	Leaf coverage is below 2% of the image

10. Results and Screenshots

The project generates `healthy_leaf.png`, `stressed_leaf.png`, and `diseased_leaf.png`. A run produces a terminal classification and an annotated PNG. Before portal submission, run the project locally and add a screenshot of your terminal output and annotated image if the evaluator expects evidence of execution.

Expected output structure

```
Classification : Mostly healthy looking
Confidence : [calculated percentage]
Leaf coverage : [calculated percentage]
Green / Yellow / Brown: [calculated values]
Annotated image: results/healthy_annotated.png
```

11. Testing Approach

Tests use controlled images in memory so expected results are repeatable.

Test	Input	Expected result
Healthy colour	Uniform green image	Mostly healthy looking; green percentage above 95%.
Brown colour	Uniform brown image	Likely diseased or severely stressed; brown above 95%.
Blank image	Light grey image	No leaf region detected.
Manual CLI	Generated sample image	Console output and annotated PNG are created.

Run tests

```
python -m unittest discover -s tests -v
```

12. Challenges Faced

Leaf colours vary with illumination, camera quality, species, and backgrounds. Selecting broadly useful HSV thresholds requires balancing false detections against missed regions. The application also needed a clean terminal workflow without a GUI or remote trained model.

13. Learnings and Key Takeaways

The project demonstrates HSV conversion, threshold masks, morphological operations, contours, feature measurement, deterministic decision rules, modular design, error handling, reproducible examples, and unit testing.

14. Future Enhancements

Future versions could use a labelled disease dataset, a convolutional neural network, learned leaf segmentation, named disease classes, precision/recall metrics, and a web interface. Such a model would require careful validation before agricultural use.

15. References

1. OpenCV Documentation, image processing and colour-space conversion.
2. NumPy Documentation, array operations.
3. Python Documentation, unittest framework.
4. Computer Vision course materials.
5. Project source code and README in the submitted GitHub repository.